Neural Networks

Comprehensive Lecture Notes

Vittorio Cava

A.Y. 2026/27

Contents

I Lecture 1 **3**

1 The Role of Mathematics in Neural Networks *(slides 3–10)* **3**

1.1 Learning outcomes *(slides 3)* . . . 3

1.2 Why mathematics? *(slides 4)* . . . 3

1.3 One computation, three viewpoints *(slides 5)* . . . 3

1.4 Fundamental mathematical disciplines for neural networks *(slides 6)* . . . 4

1.5 Two spaces that must not be confused *(slides 7)* . . . 4

1.6 Example: geometry of feature spaces *(slides 8)* . . . 5

1.7 Neural network as a composition of functions *(slides 9–10)* . . . 5

2 Linear Algebra for Neural Networks *(slides 11–26)* **6**

2.1 Introduction to Linear Algebra *(slides 11–13)* . . . 6

2.2 Algebraic Structures for Neural Networks *(slides 14–24)* . . . 7

2.3 Data Shapes *(slides 25–26)* . . . 8

3 Data Manipulation with Algebraic Operations *(slides 27–54)* **9**

3.1 Simple Algebraic Operations on Data *(slides 27–31)* . . . 9

3.2 Inner and Outer Products *(slides 32–33)* . . . 10

3.3 Dealing with Matrices *(slides 34–44)* . . . 12

3.4 Special Matrices *(slides 45–48)* . . . 16

3.5 Matrix Inversion *(slides 49–53)* . . . 17

3.6 Rank and Determinant of a Matrix *(slides 54)* . . . 20

Appendix A: Supplementary material (not on the slides) **20**

4 Summary **21**

4.1 Comparison of the operations covered . . . 21

4.2 Cost of the three “products” 22
4.3 Key take-aways 22

Part I

Lecture 1

Scope of these notes

These notes follow the flow of the slide deck (section and topic titles are kept near-verbatim; each header carries its slide range). The PDF supplied contains slides 1–54 of 103, so the notes stop where the deck stops: the end of Section 3 (*Rank and Determinant of a Matrix*). Sections 4 (*Geometric Interpretation of Linear Transformations*) and 5 (*Metrics and Norms*) are only listed in the table of contents of the slides and are **not** covered here.
No companion book chapter was supplied, so the formal statements and proofs are my own standard derivations, placed where the slides introduce each topic. Material that goes beyond what the slides contain is collected in Appendix 3.6 and labelled as supplementary. Where a slide contains a typo or an ambiguous convention, a *Remark* says so.

1 The Role of Mathematics in Neural Networks *(slides 3–10)*

1.1 Learning outcomes *(slides 3)*

By the end of the lecture (the full 103-slide deck) one should be able to:

- read and verify the *shapes* of neural-network computations;
- interpret a layer as an affine map and as a change of representation;
- use dot products, norms, projections and the SVD to reason about models;
- differentiate composed functions with gradients and Jacobians;
- explain backpropagation as reverse-mode automatic differentiation;
- connect curvature and conditioning to gradient-descent dynamics.

These notes cover the first two bullets and the algebraic tools behind the others.

1.2 Why mathematics? *(slides 4)*

Mathematics is the theoretical foundation of neural networks: it supports interpretability and reliability, and it underpins optimization and generalization. Neural networks are function approximators built from well-understood mathematical structures, so understanding those structures helps to design better architectures, train more efficiently and tackle real-world problems.

Intuition

Think of mathematics as the *type system and the map* of the field. A shape mismatch, an exploding gradient or a poorly conditioned problem is visible in the equations long before any GPU time is spent; without the maths, one is debugging by trial and error.

1.3 One computation, three viewpoints *(slides 5)*

A single step of neural-network computation is the pipeline

$$X \to Z = XW + b, \qquad Z \to H = \mathrm{ReLU}(Z), \qquad H \to \hat{Y}, \qquad \hat{Y} \to L, \qquad L \to \nabla_\theta L.$$

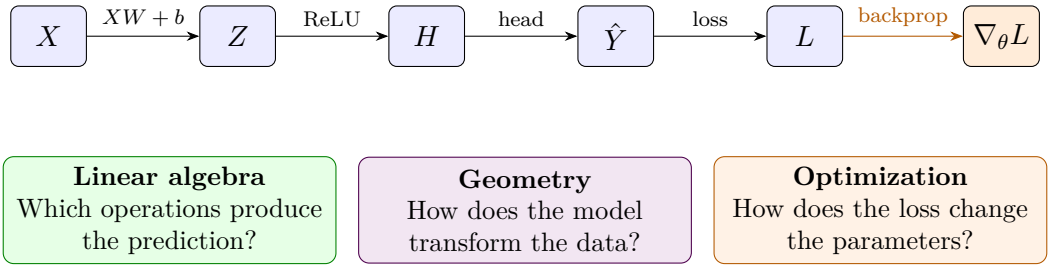


Figure 1: Redrawn from slide 5: one forward–backward step and the three questions attached to it.

The same pipeline answers three different questions, one per discipline.

Definition 1.1 (Forward step of an affine layer with ReLU)**.** Given a mini-batch $X \in \mathbb{R}^{B\times D}$, weights $W \in \mathbb{R}^{D\times H}$ and bias $b \in \mathbb{R}^H$, define

$$Z = XW + \mathbf{1}b^\mathsf{T} \in \mathbb{R}^{B\times H}, \qquad H = \mathrm{ReLU}(Z) \text{ (entrywise)},$$

followed by a prediction $\hat{Y}$ and a scalar loss $L(\hat{Y}, Y)$. The gradient $\nabla_\theta L$ collects the derivatives of L with respect to all parameters θ (here W, b and the parameters of the head).

Intuition

It is the same object seen through three lenses, like a car seen as a machine (linear algebra: which parts move), as a vehicle in space (geometry: where does it take the data) and as a driver receiving feedback (optimization: how to steer after a mistake). Each lens asks a different question, but all three are answered with the same matrix calculus.

1.4 Fundamental mathematical disciplines for neural networks *(slides 6)*

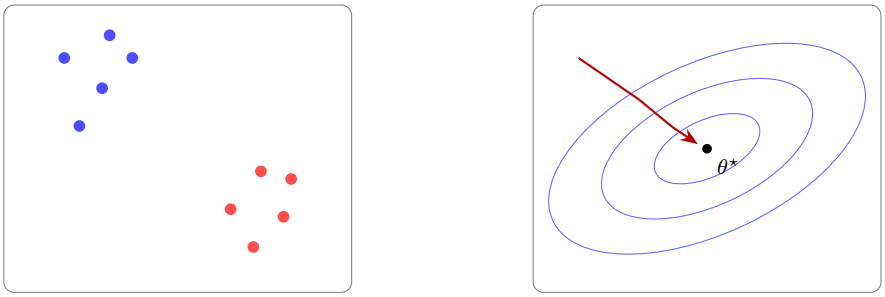
- **Linear algebra**: vectors, matrices and tensors carry all computations; factorizations such as the SVD and the eigendecomposition support dimensionality reduction.
- **Geometry**: networks act in high-dimensional spaces; manifolds, distance metrics and transformations improve robustness and interpretability.
- **Optimization**: training is a large-scale optimization problem; gradient descent, Hessians and loss analysis make it effective.

1.5 Two spaces that must not be confused *(slides 7)*

Definition 1.2 (Representation space and parameter space)**.** The *representation space* is where examples and activations live: $X \mapsto Z = f_\theta(X)$, with θ fixed. The *parameter space* is where models live: $\theta \mapsto L(\theta)$, with the data fixed.

Intuition

A neuron's activation is one coordinate of a point in the left picture, whereas its weight is one coordinate of a point in the right picture. For a dense layer $784 \to 128$, a batch of activations lives in $\mathbb{R}^{128}$ per example, but a *single model* is a point in a space with $784 \cdot 128 + 128 = 100\,480$ dimensions. Same matrix calculus, completely different questions.



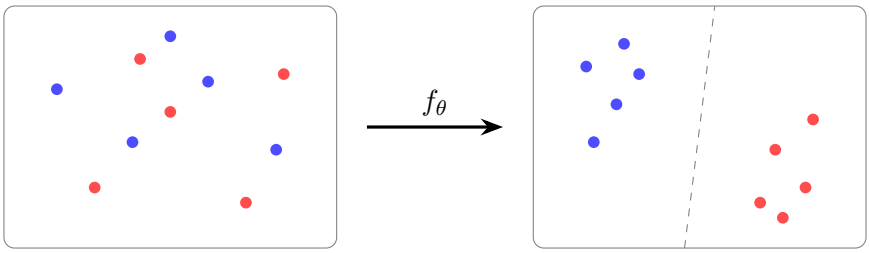
Representation space
points = examples/activations

Parameter space
points = models, contours of $L(\theta)$

Figure 2: Left: distances and directions between activations describe learned features. Right: gradients and curvature of $L(\theta)$ describe training dynamics.

1.6 Example: geometry of feature spaces *(slides 8)*

An image classifier learns to map raw inputs to a representation where classes occupy distinct regions: similar examples cluster and different classes are well separated, so that a simple decision rule on the last layer suffices.



raw inputs: classes intertwined

learned features: classes separated

Figure 3: Original figure for slide 8 (described only in prose there): the network turns an entangled input space into a feature space where a linear boundary works.

1.7 Neural network as a composition of functions *(slides 9–10)*

Definition 1.3 (Feed-forward network as a composition)**.** With weight matrices W_i (linear maps), bias vectors b_i (shifts) and an activation σ applied entrywise,

$$f_\theta(X) = W_L\,\sigma(W_{L-1}\cdots\sigma(W_1X + b_1)\cdots + b_{L-1}) + b_L. \tag{1}$$

Remark 1.4 (Column versus row convention)**.** Equation (1) multiplies X on the *left* by W_1 (data as columns, $W_1 \in \mathbb{R}^{D_{\text{out}}\times D_{\text{in}}}$), whereas the running example on slide 26 writes $Z = XW+b$ (data as rows, $W \in \mathbb{R}^{D_{\text{in}}\times D_{\text{out}}}$). The two are related by transposition, $(W_1X)^\mathsf{T} = X^\mathsf{T}W_1^\mathsf{T}$. Always check which convention a formula or library uses.

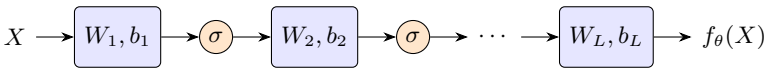


Figure 4: Equation (1) as a chain: alternating affine maps (linear algebra) and pointwise nonlinearities.

The components of (1) are: *linear algebra* (matrix products such as WX govern propagation), *nonlinearity* ($\sigma(WX)$ makes the network expressive) and *optimization* (minimizing a loss with gradient methods, e.g. $\nabla_W L$).

Central question of the course. What properties of the individual maps survive, grow or disappear under composition? This returns for expressivity, stability, gradients, recurrence and depth.

Proposition 1.5 (Without nonlinearity, depth collapses)**.** *If $\sigma = \mathrm{id}$, then f_θ in (1) is a single affine map $X \mapsto \widetilde{W}X + \tilde{b}$.*

Proof. For $L = 2$: $W_2(W_1X + b_1) + b_2 = (W_2W_1)X + (W_2b_1 + b_2)$. Induction on L: the composition of two affine maps is affine, since $W'(WX + b) + b' = (W'W)X + (W'b + b')$. □

Intuition

Stacking linear layers is like doing several successive currency conversions: however many you chain, the result is one exchange rate plus a fixed fee. Only the nonlinearity σ lets extra layers add something new. Numerically, $W_1 = \begin{bmatrix} 1 & 2 \end{bmatrix}^\top$ (a 2×1 map), $W_2 = \begin{bmatrix} 3 & -1 \end{bmatrix}$ compose to $W_2W_1 = 3 \cdot 1 + (-1) \cdot 2 = 1$: two layers, one number.

2 Linear Algebra for Neural Networks *(slides 11–26)*

2.1 Introduction to Linear Algebra *(slides 11–13)*

Fluency with vectors and matrices — multiplication, transposition, inverses — is essential for neural networks, and it is what makes backpropagation and optimization approachable. It is best learned by consistent practice; Python libraries (NumPy, PyTorch) make the operations directly executable.

Advantages of linear algebra (slide 12): compact notation; an intuitive geometric picture (transformations, feature spaces); a convenient set of operations (decompositions, optimizations); data structures suited to images, text and sequences; a universal language in the literature; and an easy equation-to-code translation.

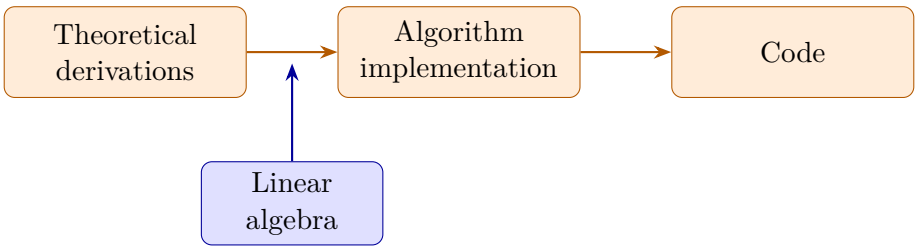


Figure 5: Redrawn from Figure 1 of the slides: linear algebra is the glue that turns theory into an algorithm whose form maps directly to code.

Intuition

Compare $y_i = \sum_j W_{ij}x_j + b_i$ written for each i with $y = Wx + b$: the second line is both the mathematical statement and (almost character by character) the NumPy line `y = W @ x + b`. That is the "equation-to-code" advantage.

2.2 Algebraic Structures for Neural Networks *(slides 14–24)*

Neural networks process data as numerical structures: *scalar* $x \in \mathbb{R}$, *vector* $\mathbf{x} \in \mathbb{R}^n$, *matrix* $\mathbf{X} \in \mathbb{R}^{M \times N}$ and *tensor* $\mathcal{X} \in \mathbb{R}^{d_1 \times \cdots \times d_n}$. Each encodes a different kind of information.

Scalars *(slides 15).* A scalar is a single real number (lowercase letter, $x \in \mathbb{R}$), e.g. the amplitude of a signal sample, one pixel value, or the numerical value of a single neuron.

Vectors *(slides 16–17).*

Definition 2.1 (Vector)**.** A vector is an ordered arrangement of scalars, denoted by a bold lowercase letter, $\mathbf{x} \in \mathbb{R}^n$, e.g. $\mathbf{x} = [1,\ 0.3,\ -2]$. Geometrically it is the path from the origin to a point P in an N-dimensional space.

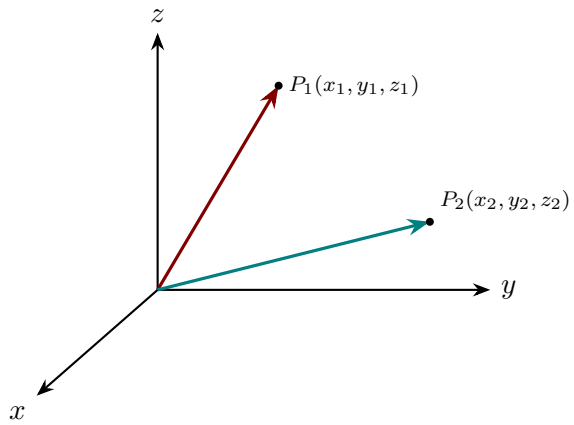


Figure 6: Redrawn from Figure 2 of the slides: vectors as arrows from the origin to points.

As input data, a vector represents a 1D digital signal (audio, brain signal, time series): an ordered sequence of samples, stored by convention as a *column* vector. (The slide writes $\mathbf{x} = [x_1\ x_2\ x_3]$ as a row for typographic economy, but states the column convention; I use columns and write $\mathbf{x}^\mathsf{T}$ for rows.) In networks, a vector may represent a layer or a sequence of weights.

Matrices *(slides 18–21).*

Definition 2.2 (Matrix)**.** A matrix $\mathbf{X} \in \mathbb{R}^{M \times N}$ is an array with M rows and N columns, $\mathbf{X} = [\mathbf{x}_1\ \cdots\ \mathbf{x}_N] = (x_{ij})$. It can be seen as a vertical stack of row vectors or a horizontal arrangement of column vectors.

As input data, matrices represent 2D signals (grayscale images) or collections of 1D signals (multichannel audio, multisensor recordings); in networks, they store weights.

Example 2.3 (Data, weights, activations *(slides 20–21)*)**.** A grayscale image is a matrix of pixel intensities; a colour image is a tensor of shape (height, width, 3). A dataset of m samples with n features is $\mathbf{X} \in \mathbb{R}^{m \times n}$ (one row per sample, one column per feature). A layer with input $\mathbf{x} \in \mathbb{R}^n$ and weights $\mathbf{W} \in \mathbb{R}^{m \times n}$ outputs $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$. For instance, with $n = 3$, $m = 2$,

$$\mathbf{W} = \begin{bmatrix} 1 & 0 & 2 \\ -1 & 1 & 0 \end{bmatrix},\ \mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix},\ \mathbf{b} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \Rightarrow\ \mathbf{y} = \begin{bmatrix} 7 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 7 \\ 2 \end{bmatrix}.$$

Tensors *(slides 22–24).*

Definition 2.4 (Tensor)**.** A tensor $\mathcal{X} \in \mathbb{R}^{d_1 \times d_2 \times \cdots \times d_n}$ is a function of three or more indices; equivalently a stacking of K matrices (slices), $\mathcal{X} = \{\mathbf{X}_1, \ldots, \mathbf{X}_K\}$, or of higher-order tensors along horizontal, vertical or lateral slices. It generalizes the matrix.

Tensors represent 3D signals (video, time-frequency maps of EEG) and higher-dimensional data (MR brain imaging).

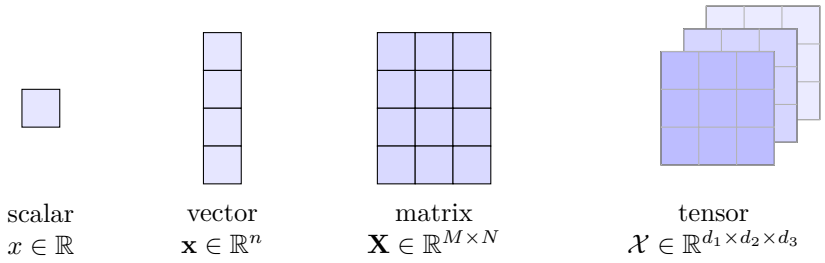


Figure 7: Original figure: scalar $\subset$ vector $\subset$ matrix $\subset$ tensor as increasing numbers of indices (0, 1, 2, 3).

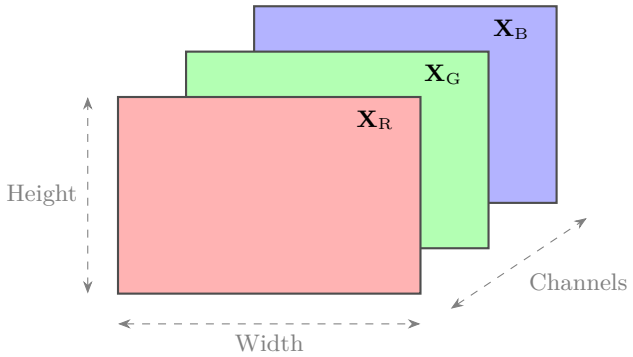


Figure 8: Redrawn from Figure 6 of the slides: a colour image as a 3D tensor $\mathcal{X} = \{\mathbf{X}_R, \mathbf{X}_G, \mathbf{X}_B\}$ of three channel matrices. (The slide labels the horizontal axis "Weight"; it is the image *width*.)

Intuition

Order of the tensor = number of indices needed to locate one scalar. A 224×224 RGB image needs three indices and holds $3 \cdot 224 \cdot 224 = 150\,528$ numbers; a batch of 32 such images needs four indices and holds $4\,816\,896$ numbers. Nothing new happens mathematically — there are just more indices to keep track of.

2.3 Data Shapes *(slides 25–26)*

Definition 2.5 (Semantics of axes *(slides 25)*)**.** The *shape* of a tensor is the tuple of its axis lengths. Its meaning comes from the *labels* of the axes, not from its order:

Object	Typical shape	Meaning
Tabular batch	$B \times D$	examples × features
Image batch	$B \times C \times H \times W$	examples × channels × space
Token batch	$B \times T \times D$	examples × tokens × features
Dense weights	$D_{\text{out}} \times D_{\text{in}}$	output × input features

Intuition

A tensor of shape $32 \times 64 \times 64$ could be 32 grayscale images, 32 sequences of 64 tokens with 64 features each, or something else entirely. Only the axis labels tell you which, and shapes will be central for CNN layouts, sequence models and multi-head attention.

Example 2.6 (Running example: one mini-batch *(slides 26)*)**.** Let $X \in \mathbb{R}^{3\times 2}$, $W \in \mathbb{R}^{2\times 4}$, $b \in \mathbb{R}^4$. The affine layer acts on every row: $Z = XW + \mathbf{1}b^\mathsf{T} \in \mathbb{R}^{3\times 4}$ (broadcasting: the same bias is added to all three examples). With

$$X = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 3 & 1 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 & -1 & 2 \\ 2 & 1 & 0 & -1 \end{bmatrix}, \quad b = \begin{bmatrix} 0.5 & -1 & 0 & 1 \end{bmatrix}$$

we get

$$XW = \begin{bmatrix} 5 & 2 & -1 & 0 \\ -2 & -1 & 0 & 1 \\ 5 & 1 & -3 & 5 \end{bmatrix}, \qquad Z = \begin{bmatrix} 5.5 & 1 & -1 & 1 \\ -1.5 & -2 & 0 & 2 \\ 5.5 & 0 & -3 & 6 \end{bmatrix}, \qquad \mathrm{ReLU}(Z) = \begin{bmatrix} 5.5 & 1 & 0 & 1 \\ 0 & 0 & 0 & 2 \\ 5.5 & 0 & 0 & 6 \end{bmatrix}.$$

E.g. $Z_{11} = 1 \cdot 1 + 2 \cdot 2 + 0.5 = 5.5$.

Algorithm 2.7 (Shape checking)**.** To verify a matrix expression before running any code: (1) write the shape of every operand with labelled axes; (2) for each product $(N \times P)(P \times M)$ check that the inner sizes agree; (3) the result has shape $N \times M$ and the shared axis has been *contracted*; (4) for a sum, check the shapes are equal or broadcastable (bias: $B \times H$ plus H).

Intuition

Shape checking is dimensional analysis for tensors: like verifying that both sides of a physics formula are in metres, it catches many implementation errors before any data is processed. In Example 2.6: $(3 \times 2)(2 \times 4) \to 3 \times 4$, the axis of size 2 is contracted, and b (length 4) is broadcast over the 3 rows.

3 Data Manipulation with Algebraic Operations *(slides 27–54)*

3.1 Simple Algebraic Operations on Data *(slides 27–31)*

Basic operations on vectors *(slides 27)*. For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ and $\lambda \in \mathbb{R}$: addition $c_i = a_i + b_i$; scalar multiplication $(\lambda \mathbf{a})_i = \lambda a_i$; and the dot product $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$, which measures the similarity of two vectors.

How algebraic operations affect data *(slides 28–29)*. Feature weighting (dot products compute the weighted sums in neurons); feature scaling (multiply each feature by a constant); feature transformation (linear maps into new feature spaces); data centering (subtract the mean); dimensionality reduction (PCA via matrix decompositions); covariance computation (matrix products); convolution (linear combinations of shifted inputs, highlighting edges, textures and shapes); and data augmentation (rotation and scaling matrices).

Algorithm 3.1 (Centering and covariance of a data matrix)**.** Given $X \in \mathbb{R}^{m\times n}$ (rows = samples): (1) $\mu = \frac{1}{m} X^\mathsf{T} \mathbf{1} \in \mathbb{R}^n$; (2) $X_c = X - \mathbf{1}\mu^\mathsf{T}$; (3) $C = \frac{1}{m-1} X_c^\mathsf{T} X_c \in \mathbb{R}^{n\times n}$.

Example 3.2. Take $X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 9 \end{bmatrix}$. Then $\mu = [3, 5]^\mathsf{T}$, $X_c = \begin{bmatrix} -2 & -3 \\ 0 & -1 \\ 2 & 4 \end{bmatrix}$ and $X_c^\mathsf{T} X_c = \begin{bmatrix} 8 & 14 \\ 14 & 26 \end{bmatrix}$, so $C = \begin{bmatrix} 4 & 7 \\ 7 & 13 \end{bmatrix}$ (feature 1 has variance 4, feature 2 has variance 13, covariance 7).

Intuition

Centering moves the data cloud so that its centre of mass sits at the origin; the covariance matrix then records, entry by entry, how strongly two features move together. Both steps are just a subtraction and one matrix product — which is why they are cheap and why PCA (next lectures) can be phrased entirely in linear algebra.

Additions and multiplications with matrices *(slides 30)*.

Definition 3.3 (Matrix sum and product)**.** For $A, B \in (\mathbb{R}, \mathbb{C})^{N \times M}$, $C = A + B$ has entries $c_{ij} = a_{ij} + b_{ij}$. For $A \in (\mathbb{R}, \mathbb{C})^{N \times P}$, $B \in (\mathbb{R}, \mathbb{C})^{P \times M}$, $C = AB \in (\mathbb{R}, \mathbb{C})^{N \times M}$ has

$$c_{ij} = \sum_{k=1}^{P} a_{ik} b_{kj}.$$

The product costs $N \cdot P \cdot M$ scalar multiplications; e.g. $(32 \times 128)(128 \times 64)$ needs $32 \cdot 128 \cdot 64 = 262\,144$.

Addition and multiplication in general algebraic structures *(slides 31)*.

Definition 3.4 (Group, ring, field)**.** A *group* $(G, +)$ is a set with an operation that is closed, associative, has an identity and inverses. A *ring* $(R, +, \cdot)$ is an abelian group under $+$ with an associative product that distributes over $+$. A *field* is a (commutative) ring in which every nonzero element has a multiplicative inverse.

Example 3.5. $\mathbb{R}$ is a field: addition and multiplication are closed and associative, every number has an additive inverse, and every nonzero number has a multiplicative inverse. The set $\mathbb{R}^{n \times n}$ is a ring but *not* a field: with $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$, $B = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$, $AB = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \neq \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} = BA$, and $N = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \neq 0$ satisfies $N^2 = 0$.

Intuition

Numbers behave so nicely that we forget they are special: one can divide, order does not matter. Matrices give up commutativity and general division, and this is exactly why the *order* of layers matters and why Sections 3.5–3.6 on inverses and rank are needed. (Floating-point numbers only approximate a field — rounding errors are the other side of "stable computations".)

3.2 Inner and Outer Products *(slides 32–33)*

Definition 3.6 (Inner and outer product)**.** For $\mathbf{u}, \mathbf{v} \in (\mathbb{R}, \mathbb{C})^N$ the inner (scalar, dot) product is

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{v}^\mathsf{H} \mathbf{u} = \sum_{i=1}^{N} u_i v_i^*.$$

For $\mathbf{u} \in (\mathbb{R}, \mathbb{C})^M$, $\mathbf{v} \in (\mathbb{R}, \mathbb{C})^N$ the outer product is the matrix

$$\mathbf{u}\mathbf{v}^{\mathsf{H}} = \begin{bmatrix} u_1 v_1^* & \cdots & u_1 v_N^* \\ \vdots & \ddots & \vdots \\ u_M v_1^* & \cdots & u_M v_N^* \end{bmatrix} \in (\mathbb{R}, \mathbb{C})^{M \times N}.$$

Remark 3.7. The slide's typeset chain of equalities for $\langle \mathbf{u}, \mathbf{v} \rangle$ is garbled in the PDF; the final sum $\sum u_i v_i^*$ is what I use, which equals $\mathbf{v}^{\mathsf{H}}\mathbf{u}$. (The other convention $\mathbf{u}^{\mathsf{H}}\mathbf{v} = \sum u_i^* v_i$ is its complex conjugate.) In the real case all conventions coincide.

Proposition 3.8. *For real vectors the inner product is bilinear and symmetric, and* $\langle \mathbf{u}, \mathbf{u} \rangle = \sum u_i^2 \geq 0$ *with equality iff* $\mathbf{u} = 0$. *The outer product* $\mathbf{u}\mathbf{v}^{\mathsf{T}}$ *has rank at most one, and* $(\mathbf{u}\mathbf{v}^{\mathsf{T}})_{ij} = u_i v_j$.

Proof. Bilinearity and symmetry follow from the definition of the sum. Positivity: a sum of squares vanishes iff every term does. Every column of $\mathbf{u}\mathbf{v}^{\mathsf{T}}$ is a multiple (v_j) of $\mathbf{u}$, so its column space is spanned by $\mathbf{u}$. □

Intuition

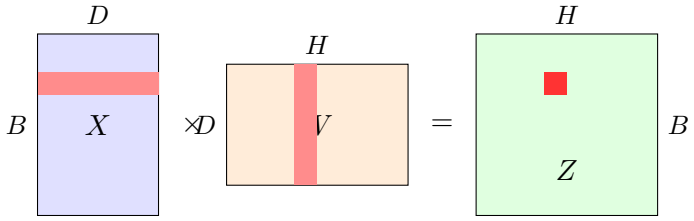
The dot product multiplies matching coordinates and adds them up: large and positive when the two vectors point the same way, zero when they are orthogonal, negative when they oppose. This is exactly what a neuron does when it weighs an input against its weights. The outer product does the opposite of collapsing: from $\mathbf{u} = [1, 2]^{\mathsf{T}}$, $\mathbf{v} = [3, 0, 1]^{\mathsf{T}}$ it builds the whole table $\begin{bmatrix} 3 & 0 & 1 \\ 6 & 0 & 2 \end{bmatrix}$ of pairwise products, a rank-one matrix. Outer products will reappear as the gradient of a linear layer.

Matrix multiplication is a contraction *(slides 33)*.

Definition 3.9 (Contraction)**.** For $X \in \mathbb{R}^{B \times D}$ and $W \in \mathbb{R}^{D \times H}$,

$$Z_{bh} = \sum_{d=1}^{D} X_{bd} W_{dh}.$$

The index d disappears because it is summed over (*contracted*); b and h remain. Rule: a *repeated* index is contracted, an *unrepeated* index appears in the output.



$Z_{bh} = \sum_d X_{bd} W_{dh}$: row b of X contracted with column h of W

Figure 9: Original figure: matrix product as a contraction of the shared axis D. The red row and column combine into one red entry.

Intuition

Reading an equation by its indices is faster than reading it by its symbols: any repeated index is "used up", anything left over is the shape of the result. The same viewpoint carries unchanged from dense layers to attention and tensor networks.

3.3 Dealing with Matrices *(slides 34–44)*

Hadamard product *(slides 34–36)*.

Definition 3.10 (Hadamard product)**.** For $A, B \in \mathbb{R}^{N\times M}$, $(A \circ B)_{ij} = a_{ij}b_{ij}$.

Unlike the matrix product it does not sum over shared indices; it is used for elementwise transformations, feature-wise scaling, attention mechanisms, weighting features in activations and elementwise normalization.

Proposition 3.11. *The Hadamard product is commutative, associative and distributes over* $+$*. If* $D_{\mathbf{a}} = \mathrm{diag}(\mathbf{a})$ *then* $D_{\mathbf{a}}\mathbf{x} = \mathbf{a} \circ \mathbf{x}$ *and* $D_{\mathbf{a}}D_{\mathbf{b}} = D_{\mathbf{a}\circ\mathbf{b}}$.

Proof. All three algebraic laws are inherited entry by entry from multiplication of real numbers. For the diagonal statement, $(D_{\mathbf{a}}\mathbf{x})_i = \sum_k (D_{\mathbf{a}})_{ik}x_k = a_i x_i$, and $(D_{\mathbf{a}}D_{\mathbf{b}})_{ij} = a_i\delta_{ij}b_j = a_i b_i \delta_{ij}$. □

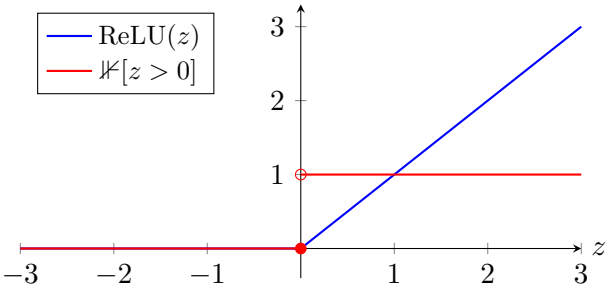


Figure 10: Original figure: ReLU and its derivative, the binary mask $\mathbb{1}[z > 0]$ (value 0 at $z = 0$ by convention).

Matrix product and Hadamard product answer different questions *(slides 36)*. The matrix product $Z = XW$ *mixes* information across the contracted feature axis; the Hadamard product $H = G \circ Z$ applies an elementwise gate or mask, for which shapes must be compatible. For ReLU the backward pass contains the binary mask

$$\frac{\partial\ \mathrm{ReLU}(Z)}{\partial Z} = \mathbb{1}[Z > 0]\,.$$

Intuition

Matrix product = a meeting where every feature talks to every other one. Hadamard product = a switchboard where each entry is individually multiplied by its own dial. In Example 2.6 the mask of Z is $\begin{bmatrix}1 & 1 & 0 & 1\\ 0 & 0 & 0 & 1\\ 1 & 0 & 0 & 1\end{bmatrix}$: during backpropagation, gradient flows only through the entries marked 1, i.e. $\partial L/\partial Z = \partial L/\partial H \circ \mathbb{1}[Z > 0]$. The same elementwise mechanism appears in LSTM gates, selective SSMs and attention masks.

Kronecker product ***(slides 37)*****.**

Definition 3.12 (Kronecker product)**.** For $A \in (\mathbb{R}, \mathbb{C})^{P \times Q}$ and $B \in (\mathbb{R}, \mathbb{C})^{N \times M}$,

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1Q}B \\ \vdots & \ddots & \vdots \\ a_{P1}B & \cdots & a_{PQ}B \end{bmatrix} \in (\mathbb{R}, \mathbb{C})^{PN \times QM}.$$

Remark 3.13. If A and B represent linear maps $T_1 : X_1 \to Y_1$ and $T_2 : X_2 \to Y_2$, then $A \otimes B$ represents the tensor product map $X_1 \otimes X_2 \to Y_1 \otimes Y_2$.

Intuition

The Kronecker product is "a copy of B scaled by each entry of A", arranged as blocks. Sizes multiply: $(2 \times 2) \otimes (3 \times 3)$ is 6×6. For $A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$, $B = \begin{bmatrix} 1 & 1 \end{bmatrix}$ the result is $\begin{bmatrix} 1 & 1 & 2 & 2 \\ 0 & 0 & 1 & 1 \end{bmatrix}$. It is the matrix form of acting with two independent maps on two independent factors (e.g. rows and columns of an image); see the Appendix for the identity connecting it to vec.

Transpose matrix *(slides 38–39).*

Definition 3.14 (Transpose)**.** For $A \in \mathbb{R}^{N \times M}$, $A^{\mathsf{T}} \in \mathbb{R}^{M \times N}$ interchanges rows and columns: $(A^{\mathsf{T}})_{ij} = a_{ji}$. Hence $(A^{\mathsf{T}})^{\mathsf{T}} = A$. Transposing a 2D signal swaps its two dimensions.

Remark 3.15. On slide 38 the printed transpose shows the entry a_{11} in position $(2, 2)$ ("$a_{12}\ a_{11}\ \cdots$"); this is a typo for a_{22}.

Hermitian matrix *(slides 40).*

Definition 3.16 (Hermitian transpose)**.** For $A \in \mathbb{K}^{N \times M}$, $A^{\mathsf{H}} = (a^*_{ji})$: transpose plus complex conjugation. In the real domain $A^{\mathsf{H}} = A^{\mathsf{T}}$.

Uses in the slides: Hermitian (self-adjoint) operators in quantum neural networks; kernels in SVMs and Gaussian processes; spectral decomposition of a Hermitian kernel matrix for dimensionality reduction.

Proposition 3.17 (Transpose rules)**.** $(AB)^{\mathsf{H}} = B^{\mathsf{H}}A^{\mathsf{H}}$, $(A + B)^{\mathsf{H}} = A^{\mathsf{H}} + B^{\mathsf{H}}$, *and* $(ABC\cdots)^{\mathsf{H}} = \cdots C^{\mathsf{H}}B^{\mathsf{H}}A^{\mathsf{H}}$.

Proof. $\left((AB)^{\mathsf{H}}\right)_{ij} = \left(\sum_k a_{jk}b_{ki}\right)^* = \sum_k b^*_{ki}a^*_{jk} = \sum_k (B^{\mathsf{H}})_{ik}(A^{\mathsf{H}})_{kj}$. The rule for sums is immediate; longer products follow by induction. □

Intuition

Transposing a product reverses the order — like taking off socks and shoes, you undo the last step first. This reversal is exactly what makes gradients travel backwards through a network: $\partial L/\partial X = \partial L/\partial Z\, W^{\mathsf{T}}$.

Matrices as row or column vectors *(slides 41–42).* The i-th row of $A \in (\mathbb{R}, \mathbb{C})^{N \times M}$ is $\mathbf{a}_i \in (\mathbb{R}, \mathbb{C})^{1 \times M}$ and its j-th column is $\mathbf{a}_j \in (\mathbb{R}, \mathbb{C})^{N \times 1}$; A is a stack of N rows or an arrangement of M columns.

Definition 3.18 (Vectorization)**.** $\mathrm{vec}(A) \in (\mathbb{R}, \mathbb{C})^{NM \times 1}$ stacks the columns of A:

$$\mathrm{vec}(A) = [a_{11}, \ldots, a_{N1},\ a_{12}, \ldots, a_{N2},\ \ldots,\ a_{1M}, \ldots, a_{NM}]^{\mathsf{H}}.$$

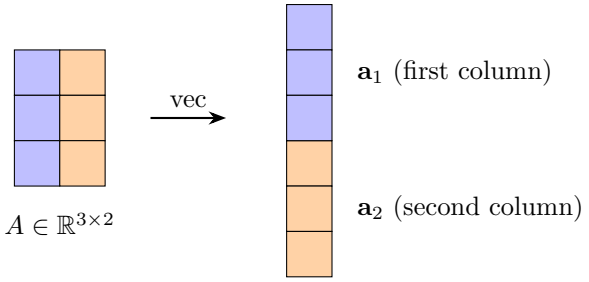


Figure 11: Original figure: vec stacks the columns of a 3×2 matrix into a 6×1 vector.

Transpose, reshape and permutation *(slides 43).* These operations can preserve values while changing how axes are interpreted:

Operation	What changes?	Typical use
X^{T}	row/column order	dot products, covariance
reshape	grouping of axes	heads, patches, flattening
permute	axis order	channel-first vs channel-last

Caution: a legal reshape need not preserve the intended semantic correspondence between axes.

Intuition

Reshaping is like re-cutting a deck of cards into different piles: the cards (values) are unchanged, but which cards sit together, and therefore what a later operation combines, has changed. Reshaping a 6×1 vector to 2×3 and 3×2 gives different matrices; only one may be the one you meant.

Exercise: shape detective *(slides 44).* Let $X \in \mathbb{R}^{32\times 128}$ and $W_Q, W_K \in \mathbb{R}^{128\times 64}$. (1) Shapes of $Q = XW_Q$ and $K = XW_K$? (2) Shape of QK^{T}? (3) Which axis is contracted in each product?

Example 3.19 (Solution)**.** (1) Both are 32×64. (2) K^{T} is 64×32, so $QK^{\mathsf{T}} \in \mathbb{R}^{32\times 32}$. (3) In XW_Q and XW_K the axis of size 128 (features) is contracted; in QK^{T} the axis of size 64 is contracted, leaving one score per pair of the 32 rows. Cost of QK^{T}: $32 \cdot 64 \cdot 32 = 65\,536$ multiplications.

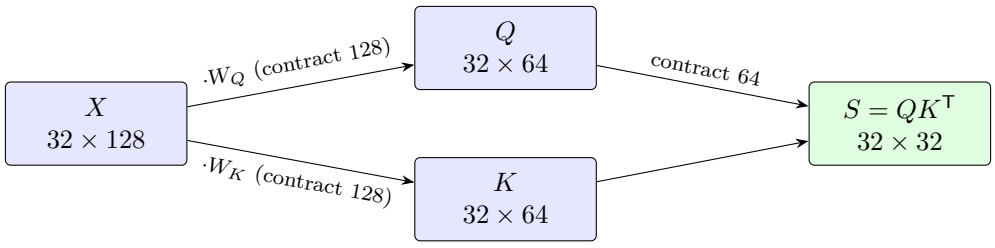


Figure 12: Original figure: shape flow of the exercise, the core of self-attention scores.

3.4 Special Matrices *(slides 45–48)*

Diagonal matrix *(slides 45).* $A \in (\mathbb{R}, \mathbb{C})^{N \times N}$ is *diagonal* if $a_{ij} = 0$ for $i \neq j$. The operator diag builds a diagonal matrix from a vector, $A = \mathrm{diag}(\mathbf{a})$, and extracts the diagonal, $\mathbf{a} = \mathrm{diag}^{-1}(A)$.

Trace and symmetry *(slides 46).*

Definition 3.20 (Trace, symmetric, Hermitian)**.** $\mathrm{tr}(A) = \sum_{i=1}^{N} a_{ii}$. A matrix is *symmetric* if $A^{\mathsf{T}} = A$ (real) and *Hermitian* if $A^{\mathsf{H}} = A$ (complex, $a_{ji} = a_{ij}^*$).

Proposition 3.21. *For $A \in \mathbb{R}^{N \times M}$, $B \in \mathbb{R}^{M \times N}$: $\mathrm{tr}(AB) = \mathrm{tr}(BA)$. For Hermitian A and any $\mathbf{w}$, $\mathbf{w}^{\mathsf{H}} A \mathbf{w} \in \mathbb{R}$.*

Proof. $\mathrm{tr}(AB) = \sum_i \sum_k a_{ik} b_{ki} = \sum_k \sum_i b_{ki} a_{ik} = \mathrm{tr}(BA)$. For the second claim, $\mathbf{w}^{\mathsf{H}} A \mathbf{w}$ is a scalar, so it equals its own conjugate transpose: $(\mathbf{w}^{\mathsf{H}} A \mathbf{w})^* = \mathbf{w}^{\mathsf{H}} A^{\mathsf{H}} \mathbf{w} = \mathbf{w}^{\mathsf{H}} A \mathbf{w}$. □

Intuition

The trace is the sum of what sits on the diagonal — a single number summarizing a square matrix, and one that does not care about the order of a product. The reality of $\mathbf{w}^{\mathsf{H}} A \mathbf{w}$ for Hermitian A is what allows us, in the next paragraph, to talk about *positive* quadratic forms at all.

Toeplitz matrix *(slides 47).*

Definition 3.22 (Toeplitz matrix)**.** $A \in (\mathbb{R}, \mathbb{C})^{N \times N}$ is Toeplitz if $a_{i,j} = a_{i+1,j+1} = a_{i-j}$: every descending diagonal is constant.

a_0	a_{-1}	a_{-2}	a_{-3}
a_1	a_0	a_{-1}	a_{-2}
a_2	a_1	a_0	a_{-1}
a_3	a_2	a_1	a_0

Figure 13: Original figure: a 4×4 Toeplitz matrix; equal shading = equal entry (a_{i-j}).

Example 3.23 (Convolution as a Toeplitz product)**.** Let $\mathbf{h} = (1, 2, 3)$ and $\mathbf{x} = (1, 0, -1)$. Their full convolution is $\mathbf{y} = (1, 2, 2, -2, -3)$, and

$$\mathbf{y} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & 2 & 1 \\ 0 & 3 & 2 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 2 \\ -2 \\ -3 \end{bmatrix}.$$

Intuition

A Toeplitz matrix is a weight matrix with a *shared* pattern: the same numbers slide along the diagonals. Convolutional layers are exactly linear layers whose weight matrix is (block-)Toeplitz, which is why they have so few free parameters (3 here instead of $5 \cdot 3 = 15$).

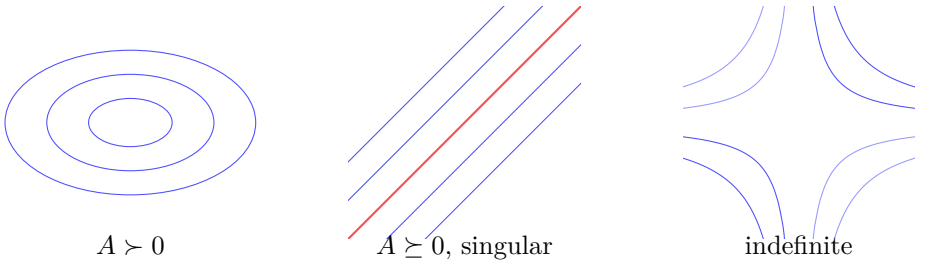
Positive semidefinite matrix *(slides 48)*.

Definition 3.24 (Positive (semi)definite)**.** $A \in \mathbb{K}^{N\times N}$ is *positive semidefinite*, $A \succeq 0$, if $\Re(\mathbf{w}^{\mathsf{H}} A\mathbf{w}) \geq 0$ for all $\mathbf{w} \in \mathbb{K}^N$; *positive definite*, $A \succ 0$, if $\mathbf{w}^{\mathsf{H}} A\mathbf{w} > 0$ for all $\mathbf{w} \neq 0$. For two matrices, $A \succeq B$ means $\mathbf{z}^{\mathsf{T}} A\mathbf{z} \geq \mathbf{z}^{\mathsf{T}} B\mathbf{z}$ for all $\mathbf{z}$.

Proposition 3.25. *For any $X \in \mathbb{R}^{m\times n}$ the matrix $X^{\mathsf{T}}X$ is positive semidefinite. In particular the covariance matrix of Algorithm 3.1 is.*

Proof. $\mathbf{w}^{\mathsf{T}} X^{\mathsf{T}} X\mathbf{w} = (X\mathbf{w})^{\mathsf{T}}(X\mathbf{w}) = \|X\mathbf{w}\|^2 \geq 0.$ □

Example 3.26. $\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$: $\mathbf{w}^{\mathsf{T}} A\mathbf{w} = w_1^2 + w_2^2 + (w_1 - w_2)^2 > 0$ for $\mathbf{w} \neq 0$, so $A \succ 0$. $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$: $(w_1 + w_2)^2 \geq 0$, zero for $\mathbf{w} = (1, -1)$: $A \succeq 0$ but not $\succ 0$. $\begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$: $\mathbf{w} = (1, -1)$ gives $1 - 4 + 1 = -2 < 0$: indefinite.



$A \succ 0$
ellipses (a bowl)

$A \succeq 0$, singular
parallel lines (a trough)

indefinite
hyperbolas (a saddle)

Figure 14: Original figure: level sets $\{\mathbf{w} : \mathbf{w}^{\mathsf{T}} A\mathbf{w} = c\}$ for the three cases of the example.

Intuition

$\mathbf{w}^{\mathsf{T}} A\mathbf{w}$ asks: "if I push in direction $\mathbf{w}$, does the landscape rise?" Positive definite means rising in every direction (a bowl, a single minimum); semidefinite allows flat directions (a trough); indefinite means some directions go down (a saddle). Covariances, Gram/kernel matrices and, later, Hessians at minima are all of this type.

3.5 Matrix Inversion *(slides 49–53)*

Inverse matrix *(slides 49–50)*.

Definition 3.27 (Inverse)**.** $A \in (\mathbb{R}, \mathbb{C})^{N\times N}$ is invertible (nonsingular) if there is B with $BA = I$, where $I = \mathrm{diag}(1, \ldots, 1)$. Then B is unique, written A^{-1}, and $A^{-1}A = I$.

Theorem 3.28 (Inverse of a product; solving $A\mathbf{x} = \mathbf{b}$)**.** *If A, B, C are invertible then $(ABC\cdots)^{-1} = \cdots C^{-1}B^{-1}A^{-1}$ and $(A^{\mathsf{H}})^{-1} = (A^{-1})^{\mathsf{H}}$. If A is nonsingular, $A\mathbf{x} = \mathbf{b}$ has the*

unique solution $\mathbf{x} = A^{-1}\mathbf{b}$.

Proof. $(AB)(B^{-1}A^{-1}) = A(BB^{-1})A^{-1} = AA^{-1} = I$; induction for longer products. Taking ^{H} of $AA^{-1} = I$ and using Proposition 3.17 gives $(A^{-1})^{\mathsf{H}}A^{\mathsf{H}} = I$. Existence: $A(A^{-1}\mathbf{b}) = \mathbf{b}$. Uniqueness: $A\mathbf{x} = \mathbf{b} \Rightarrow \mathbf{x} = A^{-1}A\mathbf{x} = A^{-1}\mathbf{b}$. □

Remark 3.29. The definition asks only for $BA = I$; for *square* matrices this implies $AB = I$ (a left inverse forces A to be injective, hence bijective in finite dimension), which justifies calling B "the" inverse. The slide's list also contains $(A+B)^{\mathsf{H}} = A^{\mathsf{H}} + B^{\mathsf{H}}$ and $(AB)^{\mathsf{H}} = B^{\mathsf{H}}A^{\mathsf{H}}$; these are transpose rules (Proposition 3.17), not inverse rules.

Intuition

Inverting a product reverses the order too, for the same reason as transposing: to undo "put on socks, then shoes" you first remove the shoes. And A^{-1} is "the undo button" of the linear map A: it exists only if A loses no information.

Pseudoinverse matrix *(slides 51–52).*

Definition 3.30 (Moore–Penrose pseudoinverse)**.** For a general (rectangular) $A \in (\mathbb{R}, \mathbb{C})^{N\times M}$, the pseudoinverse $A^{\#} \in (\mathbb{R}, \mathbb{C})^{M\times N}$ is the matrix satisfying

$$AA^{\#}A = A, \qquad A^{\#}AA^{\#} = A^{\#}, \qquad (AA^{\#})^{\mathsf{H}} = AA^{\#}, \qquad (A^{\#}A)^{\mathsf{H}} = A^{\#}A.$$

Proposition 3.31 (Uniqueness)**.** *There is at most one matrix satisfying the four conditions.*

Proof. Let X, Y both satisfy them. Then $AX = (AYA)X = AY(AX) = (AY)^{\mathsf{H}}(AX)^{\mathsf{H}} = (AXAY)^{\mathsf{H}} = (AY)^{\mathsf{H}} = AY$, and likewise $XA = X(AYA) = (XA)(YA) = (XA)^{\mathsf{H}}(YA)^{\mathsf{H}} = (YAXA)^{\mathsf{H}} = (YA)^{\mathsf{H}} = YA$. Hence $X = XAX = X(AX) = X(AY) = (XA)Y = (YA)Y = Y$. □

Theorem 3.32 (Full-rank formulas *(slides 52)*)**.** *Assume A has full rank. Then*

$$A^{\#} = \begin{cases} A^{-1} & N = M \text{ (square)}, \\ A^{\mathsf{H}}(AA^{\mathsf{H}})^{-1} & N < M \text{ ("fat")}, \\ (A^{\mathsf{H}}A)^{-1}A^{\mathsf{H}} & N > M \text{ ("tall")}. \end{cases}$$

In all three cases, if $A\mathbf{x} = \mathbf{b}$ is consistent, $\mathbf{x} = A^{\#}\mathbf{b}$ is a solution.

Proof. Tall case. Full column rank makes $A^{\mathsf{H}}A$ invertible: $A^{\mathsf{H}}A\mathbf{x} = 0 \Rightarrow \|A\mathbf{x}\|^2 = \mathbf{x}^{\mathsf{H}}A^{\mathsf{H}}A\mathbf{x} = 0 \Rightarrow A\mathbf{x} = 0 \Rightarrow \mathbf{x} = 0$. Put $A^{\#} = (A^{\mathsf{H}}A)^{-1}A^{\mathsf{H}}$. Then $A^{\#}A = I$ (Hermitian) gives conditions 2 and 4 immediately (condition 2: $A^{\#}AA^{\#} = A^{\#}$), condition 1: $AA^{\#}A = A$, and $AA^{\#} = A(A^{\mathsf{H}}A)^{-1}A^{\mathsf{H}}$ is Hermitian. The fat case is the same computation with A^{H} in place of A and AA^{H} invertible (full row rank); the square case is the ordinary inverse. Finally, if $A\mathbf{x}_0 = \mathbf{b}$, then $AA^{\#}\mathbf{b} = AA^{\#}A\mathbf{x}_0 = A\mathbf{x}_0 = \mathbf{b}$. □

Algorithm 3.33 (Solving $A\mathbf{x} = \mathbf{b}$ with the pseudoinverse)**.** (1) Read off N, M from the shape of A and check that A has full rank. (2) Pick the formula for $A^{\#}$ from Theorem 3.32 (other methods compute $A^{\#}$ via matrix decompositions). (3) Return $\mathbf{x} = A^{\#}\mathbf{b}$.

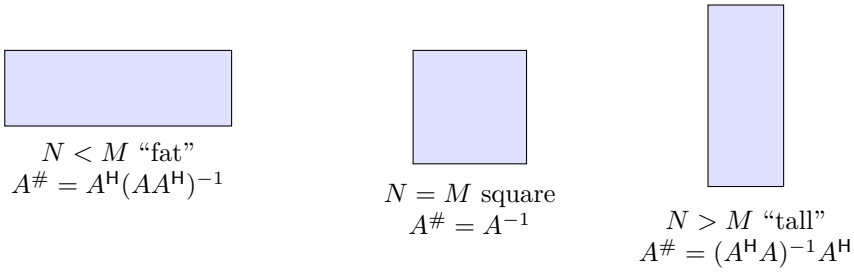


Figure 15: Original figure: the three shapes of A and the matching full-rank formulas.

Intuition

An ordinary inverse needs a square, lossless matrix. The pseudoinverse is the best available substitute for the rest: a tall matrix has more equations than unknowns (overdetermined; $A^{\#}$ is the *left* inverse, $A^{\#}A = I$), a fat one has more unknowns than equations (underdetermined; $A^{\#}$ is a *right* inverse, $AA^{\#} = I$). Example: for the tall $A = \begin{bmatrix}1\\1\end{bmatrix}$, $A^{\#} = \frac{1}{2}\begin{bmatrix}1 & 1\end{bmatrix}$, so $A^{\#}\mathbf{b}$ is the *average* of the two equations' right-hand sides. Note that the slide's formulas assume full rank; the general definition (four conditions) does not.

Matrix inversion lemma *(slides 53).*

Theorem 3.34 (Matrix inversion lemma)**.** *Let $A \in \mathbb{K}^{M\times M}$, $B \in \mathbb{K}^{M\times N}$, $C \in \mathbb{K}^{N\times N}$, $D \in \mathbb{K}^{N\times M}$, with A, C and $S = C^{-1} + DA^{-1}B$ invertible. Then*

$$[A + BCD]^{-1} = A^{-1} - A^{-1}B\big(C^{-1} + DA^{-1}B\big)^{-1}DA^{-1}.$$

Proof. Multiply the left factor by the claimed right-hand side: $(A+BCD)(A^{-1}-A^{-1}BS^{-1}DA^{-1}) = I + B\big[C - S^{-1} - CDA^{-1}BS^{-1}\big]DA^{-1}$. Since $C = CSS^{-1} = C(C^{-1} + DA^{-1}B)S^{-1} = S^{-1} + CDA^{-1}BS^{-1}$, the bracket vanishes and the product is I. □

Corollary 3.35 (Rank-one update, Sherman–Morrison)**.** *For $C = 1$, $B = \mathbf{u}$, $D = \mathbf{v}^{\mathsf{H}}$:* $(A + \mathbf{u}\mathbf{v}^{\mathsf{H}})^{-1} = A^{-1} - \dfrac{A^{-1}\mathbf{u}\mathbf{v}^{\mathsf{H}}A^{-1}}{1 + \mathbf{v}^{\mathsf{H}}A^{-1}\mathbf{u}}$.

Example 3.36. $A = 2I_2$, $\mathbf{u} = \mathbf{v} = [1,1]^{\mathsf{T}}$: $A + \mathbf{u}\mathbf{v}^{\mathsf{T}} = \begin{bmatrix}3 & 1\\1 & 3\end{bmatrix}$. Formula: $A^{-1} = \frac{1}{2}I$, $\mathbf{v}^{\mathsf{T}}A^{-1}\mathbf{u} = 1$, so the inverse is $\frac{1}{2}I - \frac{1}{2}\cdot\frac{1}{4}\begin{bmatrix}1 & 1\\1 & 1\end{bmatrix} = \frac{1}{8}\begin{bmatrix}3 & -1\\-1 & 3\end{bmatrix}$, which indeed multiplies $\begin{bmatrix}3 & 1\\1 & 3\end{bmatrix}$ to I.

Intuition

In online learning a matrix is updated by a small change every time a new sample arrives. Re-inverting an $M \times M$ matrix from scratch costs $O(M^3)$; the lemma says that if the update has low rank $N \ll M$, the new inverse is the old one plus a correction that only requires inverting an $N \times N$ matrix. This is why it is "a very useful property in the development of online machine learning algorithms".

3.6 Rank and Determinant of a Matrix *(slides 54)*

Definition 3.37 (Rank, determinant)**.** The rank $r = \operatorname{rank}(A)$ of $A \in \mathbb{R}^{M\times N}$ is the number of linearly independent columns. The determinant of a square matrix, $\det(A)$ or Δ_A, can be computed by several algorithms depending on size.

Theorem 3.38 (Solvability of linear systems)**.** *$A\mathbf{x} = \mathbf{b}$ admits a solution iff* $\operatorname{rank}(A) = \operatorname{rank}([A\ \mathbf{b}])$*, i.e. the rank of the coefficient matrix equals the rank of the complete matrix. For square A: A is invertible* $\iff \operatorname{rank}(A) = N \iff \det(A) \neq 0$.

Proof sketch. $A\mathbf{x} = \mathbf{b}$ is solvable iff $\mathbf{b}$ is a linear combination of the columns of A, i.e. iff appending $\mathbf{b}$ does not enlarge the column space, iff the two ranks agree. The square-matrix equivalences are the standard characterizations of invertibility (full rank means the columns form a basis; $\det \neq 0$ is the algebraic test for this). □

Intuition

Rank is the number of *genuinely different* directions a matrix produces. Example: $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ has rank 1 (second column $= 2\times$ first), $\det = 1\cdot 4 - 2\cdot 2 = 0$: it squashes the plane onto a line, information is lost, no inverse exists, and $A\mathbf{x} = \mathbf{b}$ is solvable only if $\mathbf{b}$ lies on that line. This is the bridge to the pseudoinverse (what to do when there is no ordinary inverse) and, in the sections not covered here, to the SVD.

Appendix A: Supplementary material *(not on the slides)*

The items below are **not** part of the slides (1–54). They support material that the slides mention and are included for completeness; they are my own standard derivations, since no companion chapter was provided.

Proposition 3.39 (Mixed product of Kronecker products; supplements slide 37)**.** $(A\otimes B)(C\otimes D) = (AC)\otimes(BD)$ *whenever the products AC and BD are defined.*

Proof. The (i,j) block of $(A\otimes B)(C\otimes D)$ is $\sum_k (a_{ik}B)(c_{kj}D) = \big(\sum_k a_{ik}c_{kj}\big)BD = (AC)_{ij}BD$, which is the (i,j) block of $(AC)\otimes(BD)$. □

Proposition 3.40 (vec and Kronecker product; links slides 37 and 42)**.** *For compatible A, X, B:* $\operatorname{vec}(AXB) = (B^{\mathsf{T}}\otimes A)\operatorname{vec}(X)$.

Proof. Let $\mathbf{x}_k$ be the columns of X and $\mathbf{b}_j$ the columns of B. The j-th column of AXB is $AX\mathbf{b}_j = A\sum_k b_{kj}\mathbf{x}_k = \sum_k b_{kj}A\mathbf{x}_k$. The j-th block row of $B^{\mathsf{T}}\otimes A$ has blocks $(B^{\mathsf{T}})_{jk}A = b_{kj}A$, so multiplying by $\operatorname{vec}(X) = [\mathbf{x}_1;\ldots;\mathbf{x}_K]$ gives exactly that sum. □

Intuition

A linear map that acts on the *columns* of X by A and on its *rows* by B is, once X is flattened with vec, a single (huge) matrix $B^{\mathsf{T}}\otimes A$. The slide's remark about the tensor product of linear maps is exactly this statement. In practice one never forms the Kronecker matrix: one computes AXB, which is far cheaper.

Proposition 3.41 (Equivalent tests for positive semidefiniteness; supplements slide 48)**.** *For real symmetric A: $A \succeq 0 \iff$ all eigenvalues are $\geq 0 \iff A = M^{\mathsf{T}}M$ for some M.*

Proof. Write $A = Q\Lambda Q^{\mathsf{T}}$ (spectral theorem, Q orthogonal). Then $\mathbf{w}^{\mathsf{T}}A\mathbf{w} = \sum_i \lambda_i (Q^{\mathsf{T}}\mathbf{w})_i^2$, which is ≥ 0 for all $\mathbf{w}$ iff every $\lambda_i \geq 0$ (choose $\mathbf{w} = Q\mathbf{e}_i$). If $\lambda_i \geq 0$, take $M = \Lambda^{1/2}Q^{\mathsf{T}}$; conversely $A = M^{\mathsf{T}}M$ is PSD by Proposition 3.25. □

Proposition 3.42 (Pseudoinverse via SVD; anticipates later lectures)**.** *If $A = U\Sigma V^{\mathsf{T}}$ is a (compact) SVD with positive singular values, then $A^{\#} = V\Sigma^{-1}U^{\mathsf{T}}$.*

Proof. With $U^{\mathsf{T}}U = V^{\mathsf{T}}V = I$: $AA^{\#}A = U\Sigma V^{\mathsf{T}}V\Sigma^{-1}U^{\mathsf{T}}U\Sigma V^{\mathsf{T}} = U\Sigma V^{\mathsf{T}} = A$; $A^{\#}AA^{\#} = A^{\#}$ likewise; $AA^{\#} = UU^{\mathsf{T}}$ and $A^{\#}A = VV^{\mathsf{T}}$ are symmetric. By Proposition 3.31 this is *the* pseudoinverse. □

Intuition

The SVD writes any matrix as "rotate, scale each axis, rotate". The pseudoinverse simply undoes each nonzero scaling and ignores the directions that were squashed to zero — the ones that carried no information and therefore cannot be recovered.

4 Summary

4.1 Comparison of the operations covered

Operation	Notation	Shapes	Sums over an index?	Role in neural networks
Dot product	$\mathbf{u}^{\mathsf{T}}\mathbf{v}$	$N, N \to$ scalar	yes, all	weighted sum in a neuron; similarity
Outer product	$\mathbf{u}\mathbf{v}^{\mathsf{T}}$	$M, N \to M \times N$	no	rank-one matrices; gradient of a linear layer
Matrix product	AB	$(N \times P)(P \times M) \to N \times M$	yes, shared axis P	linear layer XW; attention scores
Hadamard	$A \circ B$	equal shapes	no	gates, masks, ReLU derivative
Kronecker	$A \otimes B$	$(P \times Q), (N \times M) \to PN \times QM$	no	tensor product of maps
Transpose / ^{H}	A^{T}, A^{H}	$N \times M \to M \times N$	—	reverses products; backward pass
vec / reshape	$\mathrm{vec}(A)$	$N \times M \to NM \times 1$	—	flattening, heads, patches
Inverse	A^{-1}	square, full rank	—	solving $A\mathbf{x} = \mathbf{b}$; undo a lossless map
Pseudoinverse	$A^{\#}$	any $N \times M$	—	least-squares/underdetermined systems
Inversion lemma	$[A + BCD]^{-1}$	$M \times M$, low-rank update	—	online algorithms

Special matrix	Defining property	Example of where it appears
Diagonal	$a_{ij} = 0$, $i \neq j$	per-feature scaling ($D_{\mathbf{a}}\mathbf{x} = \mathbf{a} \circ \mathbf{x}$)
Symmetric / Hermitian	$A^{\mathsf{T}} = A$ / $A^{\mathsf{H}} = A$	covariance, kernels
Toeplitz	$a_{ij} = a_{i-j}$	convolution as matrix product
Positive (semi)definite	$\mathbf{w}^{\mathsf{H}}A\mathbf{w} > 0$ (≥ 0)	covariance, Gram matrices, Hessians at minima

4.2 Cost of the three "products"

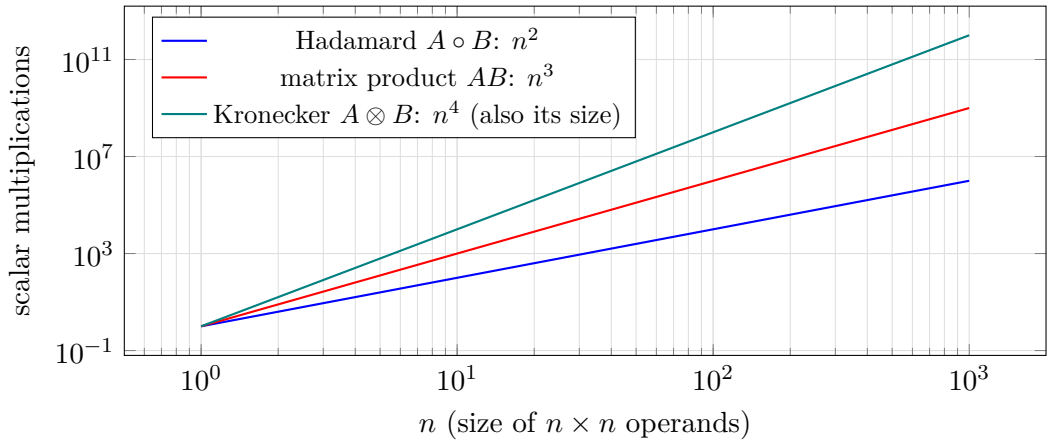


Figure 16: Original qualitative comparison: the Hadamard product touches each entry once, the matrix product mixes rows with columns, and the Kronecker product produces an $n^2 \times n^2$ matrix — which is why it is used conceptually rather than materialized.

4.3 Key take-aways

1. One forward–backward step is read through three lenses (linear algebra, geometry, optimization), and one must not confuse *representation space* (activations) with *parameter space* (models).
2. Without nonlinearity a stack of affine layers collapses to one affine map (Proposition 1.5); depth only matters because of σ.
3. Tensors get their meaning from the *labels of their axes*; matrix products are *contractions* of a shared axis, and checking shapes before coding catches most errors.
4. Matrix product mixes information across features, whereas the Hadamard product gates entries individually; the ReLU derivative is a Hadamard mask.
5. When A is not square and invertible, the pseudoinverse provides $\mathbf{x} = A^{\#}\mathbf{b}$; low-rank updates are handled efficiently by the matrix inversion lemma, and rank/determinant tell whether an inverse or a solution exists.